Python

What are we going to cover ?

language fundamentals, pvm, compiler, interpreter

Introduction to Python

function references, lambda function, first class citizen

Functions

try, except, finally, raise, else

Error Handling

installation, python package managers

Environment Configuration

list, tuple, set, dictionary

Collections

reuse code, module, packages (Boooooot)

Modules and Packages

visualize → matplotlib, plotly, seaborn
numpy, pandas,

variables, dynamically typed

Data Types

map, reduce, filter, comprehension

Functional Programming

Data Handling

arithmetic, comparison, logical, bitwise

Operators

open, save, read, copy, move

File IO

RDBMS → MySQL, NOSQL → Mongo

Database Connectivity

if-else, for-in, while, for else, while else, break

Control Flow

class, object, encapsulation, abstraction, polymorphism,

OOP

default packages → os, json, sys

REST, GraphQL, socket.io : FastAPI | Flask

Web Services

selenium → testing, web scraping
pytest → testing
playwrite
Beautiful Soup

streamlit

About Instructor

- With **18+ years of crafting technology solutions**, I bring a blend of innovation, leadership, and hands-on expertise
- As the **Associate Technical Director at Sunbeam** and a passionate **Freelance Developer**, I've explored diverse domains, solving complex problems with modern tools and technologies.
- I've had the privilege of developing **many mobile applications** that power experiences on **iOS** and **Android** platforms and crafting dynamic websites with **PHP**, **MEAN**, **and MERN stacks**
- My journey has been fuelled by a deep love for programming languages like **C, C++, Python, JavaScript, TypeScript, Go, Swift, Kotlin and PHP**
- My DevOps journey includes building **CI/CD pipelines using Jenkins, GitHub Actions & ArgoCD**, automating infrastructure with tools like **Terraform & Ansible**, leveraging cloud platforms like **AWS** to create robust, scalable systems and containerizing applications using **Docker** and orchestrating them using **Kubernetes and Docker Swarm**

Language Fundamentals

What is a computer language?

- A language is a medium to interact with computer
- A language is used to solve a problem by giving some solution (writing a program)
- Types
  - **Machine languages:** the code written in 0s and 1s and can be directly executed by CPU
  - **Assembly languages:** a language closely related to one or a family of machine languages, and which uses mnemonics to ease writing

Opcode

- Programming languages
  - **General purpose languages:** a programming language that is broadly applicable across application domains, and lacks specialized features for a particular domain

python | c | c++

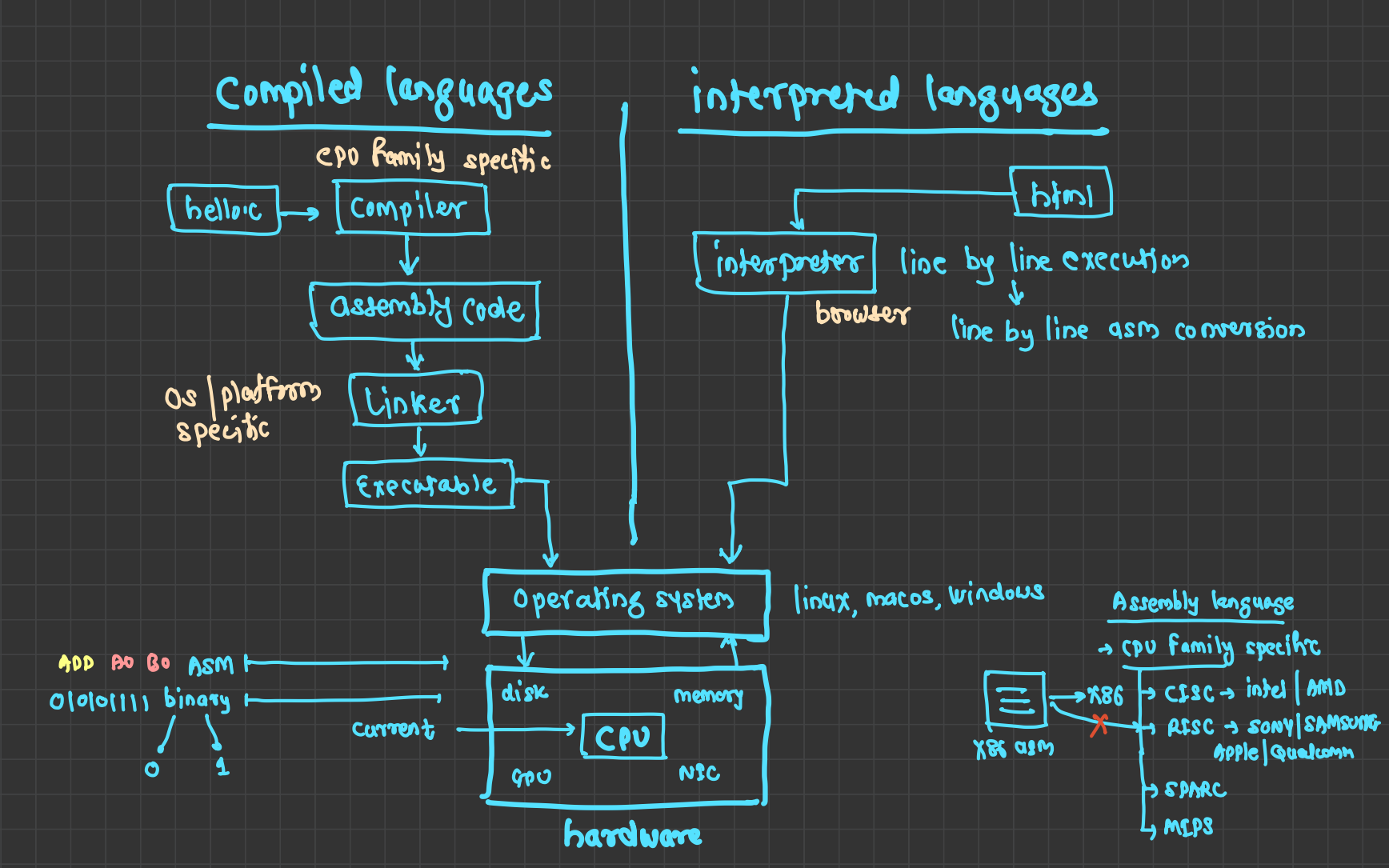
HTML, XML
  - **Markup languages:** a grammar for annotating a document in a way that is syntactically distinguishable from the text

CSS
  - **Stylesheet language:** a computer language that expresses the presentation of structured documents

json, config
  - **Configuration language:** a language used to write configuration files

SQL
  - **Query language:** a language used to make queries in databases and information systems

python, JS
  - **Scripting languages:** a language used to write simple to complex scripts


Compiled languages
interpreted languages
cpu family specific
hello.c
compiler
assembly code
linker
os | platform specific
executable
html
interpreter
line by line execution
line by line asm conversion
browser
operating system
linux, macos, windows
ADD A0 B0 ASM
01010111 binary
0
1
current
disk
memory
CPU
GPU
NIC
hardware
Assembly language
cpu family specific
x86
CISC → intel | AMD
RISC → sony | SAMSUNG
apple | Qualcomm
SPARC
MIPS
x86 asm

Low Level vs High Level Languages

High Level Language	Low Level Language
It is programmer friendly language	It is a machine friendly language
High level language is less memory efficient	Low level language is high memory efficient
It is easy to understand	It is tough to understand
It is simple to debug	It is complex to debug comparatively
It is simple to maintain	It is complex to maintain comparatively
It is portable	It is non-portable
It can run on any platform	It is machine-dependent
It needs compiler or interpreter for translation	It needs assembler for translation
E.g. Python	E.g. Assembly

Compiled language

- Compiled languages are a category of programming languages where the source code is translated into machine language or another intermediate form before it can be executed by a computer
- This translation process is typically handled by a compiler, which converts the high-level code into executable instructions that can run on a specific hardware platform
- Compiled languages generally offer better performance than interpreted languages but are less portable because the compiled code is specific to a particular architecture
- E.g. C, C++, Go, C#
- Key Characteristics
  - Faster execution speed
  - Memory efficient
  - Static Typing

Interpreted Language

- Interpreted Languages are a category of programming languages for which most of their implementations execute instructions directly and freely, without previously compiling a program into machine-language instructions
- The interpreter executes the program directly, translating each statement into a sequence of one or more subroutines and immediately carrying out the actions
- This is in contrast to compiled languages, where the code is translated into machine language before execution
- E.g. Python, JavaScript, Perl, Ruby
- Key characteristics
  - Slower execution speed compared to compiled languages
  - More portable than compiled languages
  - Easy to debug
  - Dynamic Typing
  - Memory management is simpler than compiled languages

Compiler vs Interpreter

Compiler	Interpreter
The compiler is faster, as conversion occurs before the program executes.	The interpreter runs slower as the execution occurs simultaneously.
Errors are detected during the compilation phase and displayed before the execution of a program.	Errors are identified and reported during the given actual runtime.
Compiled code needs to be recompiled to run on different machines.	Interpreted code is more portable as it can run on any machine with the appropriate interpreter.
It requires more memory to translate the whole source code at once.	It requires less memory than compiled ones.
Debugging is more complex due to batch processing of the code.	Debugging is easier due to the line-by-line execution of a code.

Python

Introduction

- **Programming Language**: Python is a high-level, general-purpose programming language
- **Created By**: Developed by Guido van Rossum and first released in 1991
- **Readability**: Known for its simple and clean syntax, emphasizing code readability → indentation
- **Interpreted**: Python is an interpreted language, meaning code is executed line by line
- **Multi-Paradigm**: Supports multiple programming styles, including:
  - Procedural
  - Object-Oriented (OOP)
  - Functional
- **Cross-Platform**: Runs on various operating systems like Windows, macOS, and Linux
- **Open Source**: Freely available and supported by a large, active community
- **Extensive Libraries**: Comes with a rich standard library and third-party packages for diverse applications
- **Versatile**: Used in web development, data science, machine learning, automation, scientific computing, and more
- **Dynamically Typed**: Variables do not require explicit type declaration

History

- Python was conceived in the late 1980s by Guido Rossum in Netherlands
- It is considered as a successor to the ABC language (itself inspired by SETL)
- Its implementation began in December 1989
- Van Rossum continued as Python's lead developer until July 12, 2018
- When he announced his permanent vacation from his responsibilities, a team of five members was developed in Jan 2019 to lead the project

Key Features

- Easy to Learn and Use
  - Simple and readable syntax, making it beginner-friendly
  - Emphasizes code readability with indentation and minimalistic design
- Interpreted Language
  - Code is executed line by line, without the need for compilation
  - Enables rapid development and testing
- Cross-Platform Compatibility
  - Runs on multiple operating systems, including Windows, macOS, and Linux
- Dynamically Typed
  - No need to declare variable types explicitly; types are inferred at runtime
- Extensive Standard Library
  - Comes with a rich set of built-in modules for tasks like file I/O, regular expressions, and web development
- Supports Multiple Programming Paradigms
  - Procedural, object-oriented, and functional programming styles

Differences between Python and C Language

C Language	Python
C is procedure-oriented programming language. It does not contain the features like classes, objects, inheritance, polymorphism, etc.	Python is object-oriented oriented language. It contains features like classes, objects, inheritance, polymorphism, etc.
Pointers concept is available in C	Python does not use pointers
C has switch statement	Python does not have switch statement
C programs execute faster	Python programs are slower compared to C. PyPy flavor or Python programs run a bit faster but still slower than C
Compulsory to declare the datatypes of variables, arrays etc	Type declaration is not required in Python
C does not have exception handling facility and hence C programs are weak	Python handles exceptions and hence Python programs are robust
C does not contain a garbage collector	Automatic garbage collector is available in Python
C supports in-line assignments	Python does not support in-line assignments
Indentation of statements is not necessary in C	Indentation is required to represent a block of statements

Versions

- Version 0.9.0 [Feb 1991]
  - Having features like classes with inheritance, exception handling, functions etc.
  - One of the major versions of python
- Version 1 [Jan 1994]
  - The major new features included in this release were the functional programming tools like labda, map, filter, reduce
  - The last version released was 1.6 in 2000
- Version 2 [Oct 2000] → deprecated
  - Introduced features like list comprehension, garbage collection, generators etc.
  - Introduced its own license known as Python Software Foundation License (PSF)
  - The last version released was 2.7.16 in Mar 2019
- Version 3 [Dec 2008]
  - Python 3.0 is also called "Python 3000" or "Py3K"
  - It was designed to rectify fundamental design flaws in the language
  - Python 3.0 had an emphasis on removing duplicative constructs and modules
  - The last version released was 3.7.4 in Jul 2019

Environment Setup

- Ubuntu (Linux)
  - Install python3 on using apt installer
    - sudo apt-get install python3 python3-pip
  - Download VS Code for Linux
    - https://code.visualstudio.com/download
    - Install python extension for code IntelliSense
- Windows
  - Download the python from
    - https://www.python.org/downloads/
  - Download VS Code for Windows
    - https://code.visualstudio.com/download
    - Install python extension for code IntelliSense

Python Implementations

- Python provides language specifications
- An **implementation** is the actual software you download and run that reads your Python code, translates it, and executes it according to those rules

Implementation	Language	Key Feature	Best Use Case	Python 3?
CPython	C	Standard, best library support	General use, data science, production	Yes
PyPy	RPython	JIT Compiler (very fast)	Long-running pure-Python apps	Yes
Jython	Java	Compiles to JVM bytecode	Integrating with Java (Legacy)	No (2.7)
IronPython	C#	Integrates with .NET / C#	Windows desktop apps (Legacy)	Partial
GraalPy	Java	High-performance JVM JIT	Modern polyglot (Java+Python)	Yes
MicroPython	C	Ultra-lightweight	Microcontrollers / Embedded	Partial

CPython : both compiler & interpreter are written in C language

- CPython is the reference implementation of the Python programming language
- It is written in C and serves as the standard interpreter for executing Python code
- CPython is an essential component of the broader Python ecosystem and provides the core functionality that developers interact with when working with Python
- Key Features

default python.org site

  - Reference Implementation: CPython is the official interpreter for Python and serves as a reference implementation against which other implementations (like Jython, IronPython, and PyPy) are compared.
  - Written in C: The core components of CPython, including the Python interpreter and built-in modules, are written in C. This allows for efficient implementation and integration with system-level operations.
  - Interpreted Language: Despite being written in C, Python is an interpreted language. CPython interprets the high-level Python source code at runtime and converts it into bytecode, which is then executed on the virtual machine.
  - C API: CPython provides a C Application Programming Interface (API), enabling developers to write Python extensions in C. This is useful for optimizing performance-critical parts of applications.

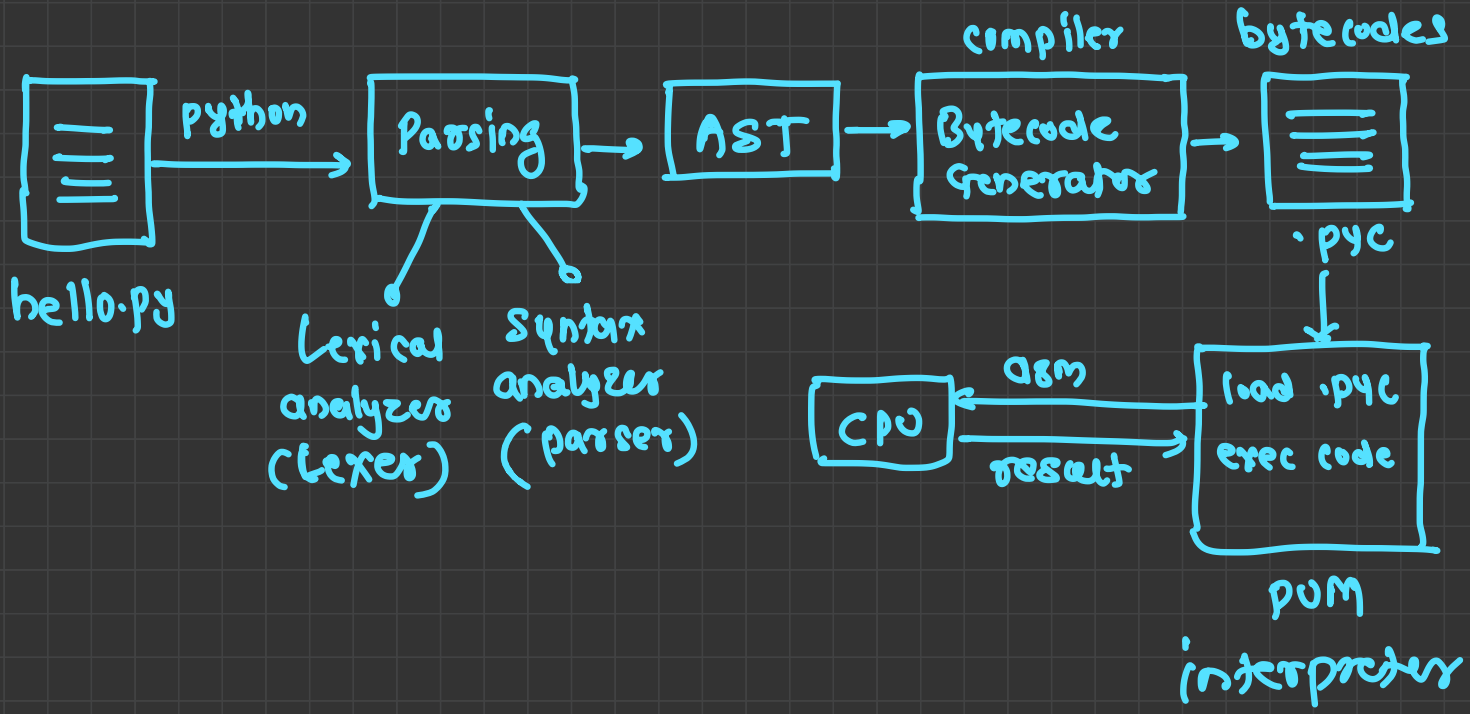
CPython Process

- **Stage 1: The Parser (Lexing & Parsing)**
  - When you run your script, CPython first reads your .py file as a plain stream of characters
  - **Lexical Analysis (Lexer):** The CPython lexer scans your text and breaks it into meaningful tokens (keywords like if, def, operators like +, and variable names)
  - **Syntax Analysis (Parser):** It takes these tokens and checks if they follow Python's grammar rules. If you forget a colon (:) or have a mismatched parenthesis, the parser throws a **SyntaxError** immediately, and your program stops before it even begins.
  - If the syntax is correct, the parser builds a tree-like structure called an **Abstract Syntax Tree (AST)**
- **Stage 2: The Compiler (Bytecode Generation)**
  - CPython takes the AST and compiles it into **bytecode**
  - Bytecode is a low-level, platform-independent set of instructions. It is **not** machine code (your CPU cannot read it). Think of it as a middle-ground between your Python code and your computer's hardware
  - These instructions look like LOAD_FAST (to load a variable), CALL_FUNCTION (to call a function), or BINARY_OP (to add numbers)
  - By default, CPython does **not** save this bytecode to a file unless it detects that it can (which it does by creating a __pycache__/your_script.cpython-312.pyc file). This way, if you run the script again without changing it, CPython can skip this compilation stage next time.

↳ asm code for PVM

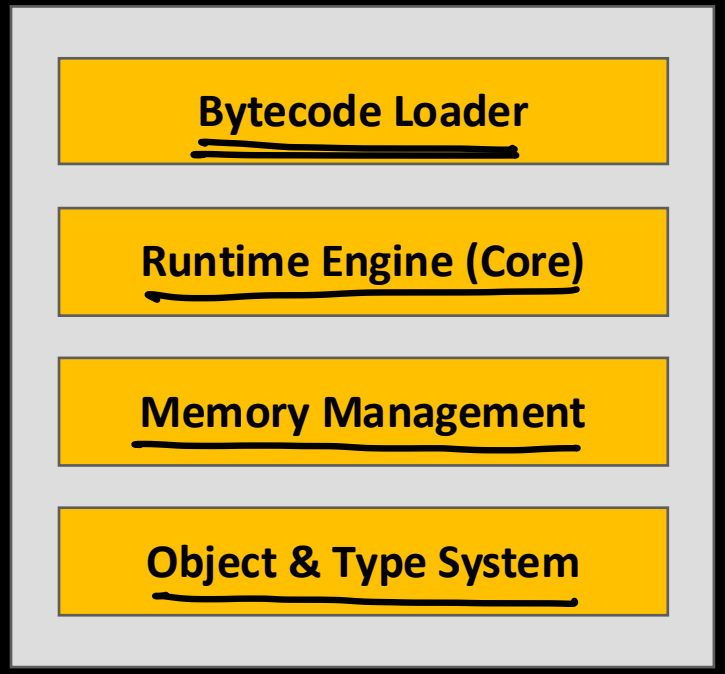
CPython Process

- **Stage 3: The Interpreter (The Python Virtual Machine)**
  - This is the heart of CPython. The compiled bytecode is fed into the **Python Virtual Machine (PVM)**—which is actually just a massive loop (called the ceval loop, short for "C evaluation") written in C
  - It reads the bytecode instructions one by one, in order
  - For each instruction, it performs a C function that executes that operation
  - For example, if the bytecode is BINARY_OP +, the PVM calls a C function that takes two numbers off the stack, adds them together using your CPU's arithmetic logic, and pushes the result back onto the stack
  - **Crucially, the PVM does not translate your code to C.** Your Python code never becomes a .c file. Instead, C is used to build the "engine" that drives your Python code forward
- **Stage 4: The Runtime & Memory Management**
  - While the interpreter runs your bytecode, two critical background processes are constantly happening
  - **The Global Interpreter Lock (GIL):** This is a mutex (a lock) inside CPython that ensures only one thread executes Python bytecode at a time. It protects CPython's memory management from crashing, but it means that CPU-heavy multithreading is limited.
  - **The Garbage Collector & Reference Counter:** As your code creates variables, lists, and objects, CPython keeps a count of how many references point to each object. When that count hits zero, CPython immediately frees up that memory. It also runs a cyclic garbage collector to clean up objects that reference each other in a loop


python
Parsing
AST
compiler
Bytecode Generator
bytecodes
.pyc
hello.py
Lexical analyzer (lexer)
Syntax analyzer (parser)
asm
result
CPU
load .pyc
exec code
PVM
interpreter

PVM

- **PVM** stands for **Python Virtual Machine**
- It's the heart of CPython—the runtime engine that actually executes your Python code
- Instead, it's a **bytecode interpreter**—a program that reads and executes Python bytecode instructions
- In CPython, the PVM is implemented in **C** as a massive loop called the **ceval loop** (short for "C evaluation")
- In PVM's interpretation model
  - **Each bytecode instruction** requires multiple C-level operations
  - **No JIT compilation** (unlike PyPy or Java)
  - **Dynamic typing** means type checks happen at runtime
- This trade-off gives Python
  - **Portability:** Same bytecode runs everywhere
  - **Flexibility:** Dynamic features like monkey-patching, eval, and dynamic attribute access
  - **Simplicity:** Easier to implement and maintain



PVM Architecture

Foundation

Identifier and keywords

- Identifiers
  - Identifiers or symbols are the names you supply for variables, types, functions, and classes
  - Identifier names must differ in spelling and case from any keywords
- Keywords
  - Keywords are the identifiers which are reserved
  - They can not be used to naming variables, types, functions or classes
  - There are 35 keywords in python 3.8
    - Value Keywords: True, False, None
    - Operator Keywords: and, or, not, in, is
    - Control Flow Keywords: if, elif, else
    - Iteration Keywords: for, while, break, continue, else
    - Structure Keywords: def, class, with, as, pass, lambda
    - Returning Keywords: return, yield
    - Import Keywords: import, from, as
    - Exception-Handling Keywords: try, except, raise, finally, else, assert
    - Asynchronous Programming Keywords: async, await
    - Variable Handling Keywords: del, global, nonlocal

Rules and Conventions

- Constant and variable names should have a combination of letters in lowercase (**a to z**) or uppercase (**A to Z**) or digits (**0 to 9**) or an underscore (**_**)
  - If you want to create a variable name having two words, use underscore to separate them
- Create a name that makes sense (use meaningful names)
- Use capital letters possible to declare a constant
- Never use special symbols like !, @, #, $, %, etc.
- Don't start a variable name with a digit
- Identifiers are case sensitive

Variable

- A variable is a named location used to store data in the memory
- It is helpful to think of variables as a container that holds data that can be changed later
- E.g.
  - value = 20
  - name = "steve"

Constants

- Constants are written in all capital letters and underscores separating the words
- E.g.
  - PI = 3.14

- In reality, we don't use constants in Python
- Naming them in all capital letters is a convention to separate them from variables, however, it does not actually prevent reassignment

Statements

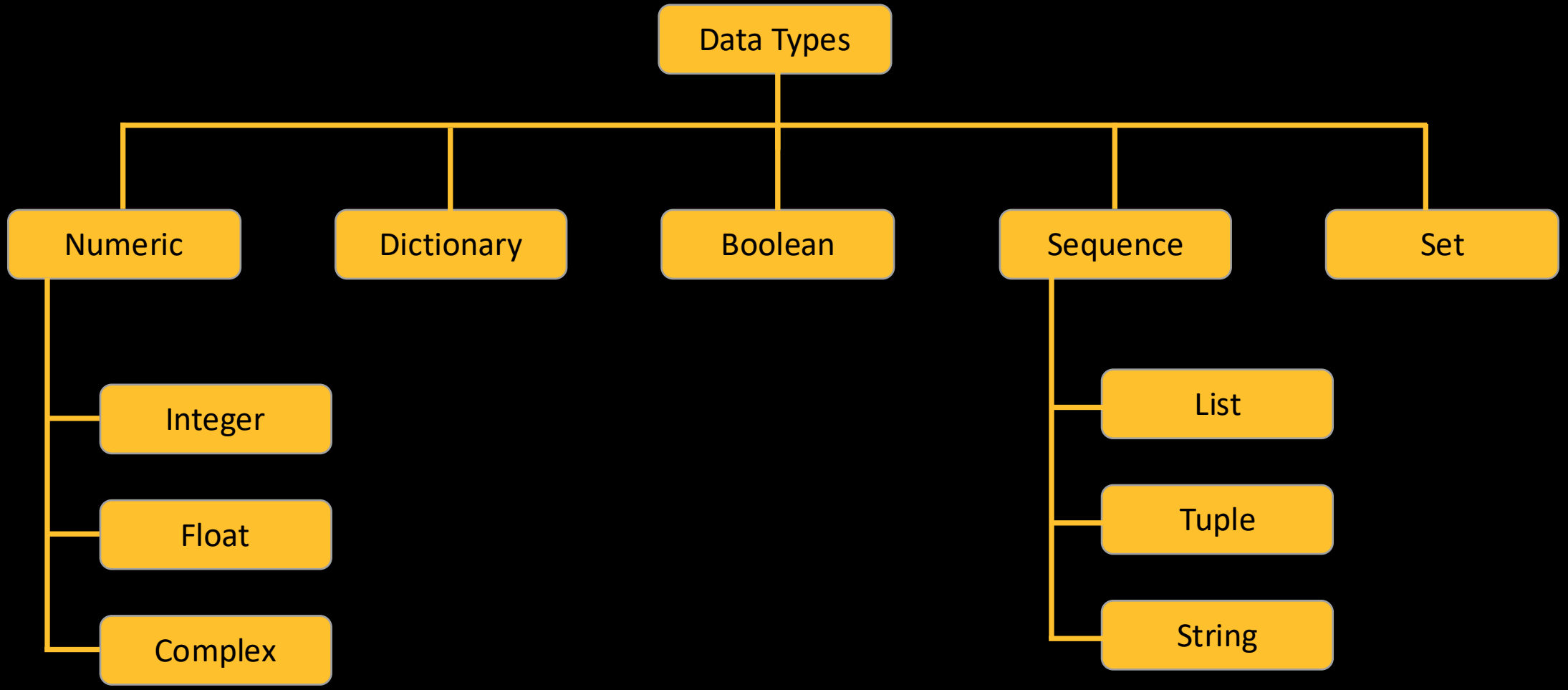
- Instructions that a Python interpreter can execute are called statements
  - E.g., a = 1 is an assignment statement
- **Single line statements**
  - The end of a statement is marked by a newline character
- **Multi-line statement**
  - We can make a statement extend over multiple lines with the line continuation character (\)
  - Line continuation is implied inside parentheses ( ), brackets [ ], and braces { }
  - We can also put multiple statements in a single line using semicolons
- **Comments**
  - Comments are very important while writing a program
  - They describe what is going on inside a program, so that a person looking at the source code does not have a hard time figuring it out
  - In Python, we use the hash (#) symbol to start writing a comment

Data Types

Need of Data Types

- Data type is an attribute associated with a piece of data that tells a computer system how to interpret its value
- Understanding data types ensures that data is collected in the preferred format and the value of each property is as expected
- It helps to know what kind of operations are often performed on a variable

Python Data Types



Literals

- Literal is a raw data given in a variable or constant

- **Numeric Literals**
  - Numeric Literals are immutable (unchangeable)
  - Numeric literals can belong to 3 different numerical types:
    - Integer (value = 100)
    - Float (salary = 15.50)
    - Complex (x = 10 + 5j)

- **String literals**
  - A string literal is a sequence of characters surrounded by quotes
  - We can use both single, double, or triple quotes for a string
  - E.g., name = “steve” or name = ‘steve’

Literals

- **Boolean literals**
  - A Boolean literal can have any of the two values: True or False
  - E.g., can_vote = True or can_vote = False

- **Special literal**
  - Python contains one special literal i.e. None
  - We use it to specify that the field has not been created
  - E.g., value = None

- **Literal Collections**
  - Used to declare variables of collection types
  - There are four different literal collections List literals, Tuple literals, Dict literals, and Set literals

Type Hinting

- Due to the dynamic nature of Python, inferring or checking the type of an object being used is especially hard
- This fact makes it hard for developers to understand what exactly is going on in code they haven't written. As a result they resort to trying to infer the type with around 50% success rate.
- **Why type hints?**
  - **Helps type checkers:** By hinting at what type you want the object to be the type checker can easily detect if, for instance, you're passing an object with a type that isn't expected
  - **Helps with documentation:** A third person viewing your code will know what is expected where, ergo, how to use it without getting them TypeErrors
  - **Helps IDEs develop more accurate and robust tools:** Development Environments will be better suited at suggesting appropriate methods when know what type your object is

Type Conversion

- The process of converting the value of one data type (integer, string, float, etc.) to another data type is called type conversion
- Python has two types of type conversion.
  - Implicit Type Conversion
    - Python automatically converts one data type to another data type
    - This process doesn't need any user involvement
    - E.g.
      - result = 10 + 35.50
  - Explicit Type Conversion
    - Users convert the data type of an object to required data type
    - We use the predefined functions like int(), float(), str(), etc to perform explicit type conversion
    - E.g.
      - num_str = str(1024)

Type Conversion

- Type Conversion is the conversion of object from one data type to another data type
- Implicit Type Conversion is automatically performed by the Python interpreter
- Python avoids the loss of data in Implicit Type Conversion
- Explicit Type Conversion is also called Type Casting, the data types of objects are converted using predefined functions by the user
- In Type Casting, loss of data may occur as we enforce the object to a specific data type

Operators

Operators

- Operators are special symbols in Python that carry out arithmetic or logical computation
- The value that the operator operates on is called the operand
- Types
  - Arithmetic operators
  - Comparison operators
  - Logical operators
  - Bitwise operators
  - Special operators
  - Membership operators

Arithmetic Operators

Operator	Meaning	Example
+	Add two operands or unary plus	x + y + 2
-	Subtract right operand from the left or unary minus	x – y
*	Multiply two operands	x * y
/	Divide left operand by the right one (always results into float)	x / y
%	Modulus - remainder of the division of left operand by the right	x % y
//	Floor division - division that results into whole number adjusted to the left in the number line	x // y
**	Exponent - left operand raised to the power of right	x**y

Arithmetic Operators

- Precedence
  - Decides how the expression will be solved
  - Brackets ()
  - Power Of (**)
  - Multiplication (*)
  - True Division (/)
  - Floor Division (//)
  - Addition (+)
  - Subtraction (-)
- Associativity
  - When there are two ore more operators in an expression with same precedence then associativity is used to decide the way to solve the expression
  - All the operators have left to right associativity except power operator (which is right to left)

Comparison operators

Operator	Meaning	Example
>	Greater than - True if left operand is greater than the right	x > y
<	Less than - True if left operand is less than the right	x < y
==	Equal to - True if both operands are equal	x == y
!=	Not equal to - True if operands are not equal	x != y
>=	Greater than or equal to - True if left operand is greater than or equal to the right	x >= y
<=	Less than or equal to - True if left operand is less than or equal to the right	x <= y

Logical operators

Operator	Meaning	Example
and	True if both the operands are true	x and y
or	True if either of the operands is true	x or y
not	True if operand is false (complements the operand)	not x

Assignment Operators

Operator	Meaning	Example
=	x = 5	x = 5
+=	x += 5	x = x + 5
-=	x -= 5	x = x - 5
*=	x *= 5	x = x * 5
/=	x /= 5	x = x / 5
%=	x %= 5	x = x % 5
//=	x //= 5	x = x // 5

Operator	Meaning	Example
**=	x **= 5	x = x ** 5
&=	x &= 5	x = x & 5
|=	x |= 5	x = x | 5
^=	x ^= 5	x = x ^ 5
>>=	x >>= 5	x = x >> 5
<<=	x <<= 5	x = x << 5

Bitwise operators

Operator	Meaning	Example
&	Bitwise AND	x & y = 0 (0000 0000)
|	Bitwise OR	x | y = 14 (0000 1110)
~	Bitwise NOT	~x = -11 (1111 0101)
^	Bitwise XOR	x ^ y = 14 (0000 1110)
>>	Bitwise right shift	x >> 2 = 2 (0000 0010)
<<	Bitwise left shift	x << 2 = 40 (0010 1000)

Special operators

Operator	Meaning	Example
is	True if the operands are identical (refer to the same object)	x is True
is not	True if the operands are not identical (do not refer to the same object)	x is not True

Membership Operators

Operator	Meaning	Example
in	True if value/variable is found in the sequence	5 in x
not in	True if value/variable is not found in the sequence	5 not in x

Functions

Functions

- In Python, a function is a group of related statements that performs a specific task
- Functions help break our program into smaller and modular chunks
- As our program grows larger and larger, functions make it more organized and manageable
- Furthermore, it avoids repetition and makes the code reusable
- Syntax

```
def function_name(parameters):
    """docstring"""
    statement(s)
```

Docstrings

- The first string after the function header is called the docstring and is short for documentation string
- It is briefly used to explain what a function does
- Although optional, documentation is a good programming practice
- We generally use triple quotes so that docstring can extend up to multiple lines
- This string is available to us as the __doc__ attribute of the function

Function Types

- Functions can divided into following two types:
  - **Built-in functions**
    - Functions that readily come with Python are called built-in functions
    - If we use functions written by others in the form of library, it can be termed as library functions
    - E.g., str(), int(), float() etc.
  - **User defined functions**
    - Functions that we define ourselves to do certain specific task are referred as user-defined functions
    - User-defined functions help to decompose a large program into small segments which makes program easy to understand, maintain and debug
    - If repeated code occurs in a program. Function can be used to include those codes and execute when needed by calling that function.
    - Programmers working on large project can divide the workload by making different functions

Returning a value

- The return statement is used to exit a function and go back to the place from where it was called
- This statement can contain an expression that gets evaluated and the value is returned
- If there is no expression in the statement or the return statement itself is not present inside a function, then the function will return the None object

Scope of variables

- Scope of a variable is the portion of a program where the variable is recognized
- Parameters and variables defined inside a function are not visible from outside the function. Hence, they have a local scope.
- The lifetime of a variable is the period throughout which the variable exists in the memory
- The lifetime of variables inside a function is as long as the function executes
- They are destroyed once we return from the function. Hence, a function does not remember the value of a variable from its previous calls.

Local Scope

- A variable declared inside the function's body or in the local scope is known as a local variable

```
def foo():
    local_var = "local"

foo()
# error
print(local_var)
```

Global Scope

- In Python, a variable declared outside of the function or in global scope is known as a global variable
- This means that a global variable can be accessed inside or outside of the function

```
g_var = "global"
def foo():
    print(“inside foo”)
    print(g_var)

foo()
```

Global Keyword

- In Python, global keyword allows you to modify the variable outside of the current scope
- It is used to create a global variable and make changes to the variable in a local context
- **Rules of global Keyword**
  - When we create a variable inside a function, it is local by default
  - When we define a variable outside of a function, it is global by default. You don't have to use global keyword
  - We use global keyword to read and write a global variable inside a function
  - Use of global keyword outside a function has no effect

Nested Function

- Function within a function is called as nested function or inner function
- E.g.

```
def outer():
    print("inside outer")
    def inner():
        print("inside inner")
    inner()

outer()
# error
inner()
```

Anonymous/Lambda Function

- In Python, an anonymous function is a function that is defined without a name
- While normal functions are defined using the def keyword in Python, anonymous functions are defined using the lambda keyword
- Hence, anonymous functions are also called lambda functions
- Syntax
  - lambda arguments: expression
- Characteristics
  - It can only contain expressions and can’t include statements in its body
  - It is written as a single line of execution
  - It does not support type annotations
  - It can be immediately invoked

Collections

List

- Python lists are one of the most versatile data types that allow us to work with multiple elements at once
- For example
  - # a list of programming languages
  - ['Python', 'C++', 'JavaScript']
- List is created by placing elements inside square brackets [], separated by commas
- List is **mutable**

List - functions

- append()
  - adds an element to the end of the list
- extend()
  - adds all elements of a list to another list
- insert()
  - inserts an item at the defined index
- remove()
  - removes an item from the list
- pop()
  - returns and removes an element at the given index
- clear()
  - removes all items from the list
- index()
  - returns the index of the first matched item
- count()
  - returns the count of the number of items passed as an argument
- sort()
  - sort items in a list in ascending order
- reverse()
  - reverse the order of items in the list
- copy()
  - returns a shallow copy of the list

Accessing List Members

- We can use the index operator [] to access an item in a list
- In Python, indices start at 0. So, a list having 5 elements will have an index from 0 to 4
- Trying to access indexes other than these will raise an **IndexError**
- The index must be an integer. We can't use float or other types, this will result in **TypeError**
- **Negative indexing**
  - Python allows negative indexing for its sequences
  - The index of -1 refers to the last item, -2 to the second last item and so on

List Slicing

- We can access a range of items in a list by using the slicing operator

```
my_list = ['p','r','o','g','r','a','m','i','z']

# elements from index 2 to index 4
print(my_list[2:5])

# elements from index 5 to end
print(my_list[5:])

# elements beginning to end
print(my_list[:])
```

Tuple

- A tuple is a collection of objects which ordered and immutable
- Tuples are sequences, just like lists
- The differences between tuples and lists are
  - Tuples cannot be changed unlike lists
  - Tuples use parentheses, whereas lists use square brackets

Tuple Operations

- Creating Tuples
- Concatenation of Tuples
- Nesting of Tuples (Tuple of Tuples)
- Repetition of Tuples
- Packing and Unpacking
- Indexing in Tuple
- Slicing in Tuple

Set

- A Python set is the collection of the unordered items
- Each element in the set must be unique
- Sets are mutable which means we can modify it after its creation
- Unlike other collections in Python, there is no index attached to the elements of the set, i.e., we cannot directly access any element of the set by the index
- However, we can print them all together, or we can get the list of elements by looping through the set

Set Operations

- Union of two Sets
- Intersection of two sets
- Difference between the two sets
- Frozenset
- Symmetric Difference of two sets

Dictionary

- Python Dictionary is used to store the data in a key-value pair format
- The dictionary is the data type in Python, which can simulate the real-life data arrangement where some specific value exists for some particular key
- It is the mutable data-structure
- The dictionary is defined into element Keys and values
- Keys must be a single element
- Value can be any type such as list, tuple, integer, etc.
- In other words, we can say that a dictionary is the collection of key-value pairs where the value can be any Python object

Dictionary Operations

- Creating the dictionary
- Accessing the dictionary values
- Adding dictionary values
- Deleting elements using del keyword

String

String

- A string is a data structure in Python that represents a sequence of characters
- It is an immutable data type, meaning that once you have created a string, you cannot change it
- Strings are used widely in many different applications, such as storing and manipulating text data, representing names, addresses, and other types of data that can be represented as text
- E.g. name = “steve”

- Python does not have a character data type, a single character is simply a string with a length of 1. Square brackets can be used to access elements of the string
- E.g. ch= “a”
-

Advantages of String in Python

- Strings are used at a larger scale i.e. for a wide areas of operations such as storing and manipulating text data, representing names, addresses, and other types of data that can be represented as text
- Python has a rich set of string methods that allow you to manipulate and work with strings in a variety of ways. These methods make it easy to perform common tasks such as converting strings to uppercase or lowercase, replacing substrings, and splitting strings into lists.
- Strings are immutable, meaning that once you have created a string, you cannot change it. This can be beneficial in certain situations because it means that you can be confident that the value of a string will not change unexpectedly.
- Python has built-in support for strings, which means that you do not need to import any additional libraries or modules to work with strings. This makes it easy to get started with strings and reduces the complexity of your code.
- Python has a concise syntax for creating and manipulating strings, which makes it easy to write and read code that works with strings.

Escape Sequencing in Python

- While printing Strings with single and double quotes in it causes SyntaxError because String already contains Single and Double Quotes and hence cannot be printed with the use of either of these
- Hence, to print such a String either Triple Quotes are used or Escape sequences are used to print such Strings
- Escape sequences start with a backslash and can be interpreted differently
- If single quotes are used to represent a string, then all the single quotes present in the string must be escaped and same is done for Double Quotes

Drawbacks of String in Python

- When we are dealing with large text data, strings can be inefficient. For instance, if you need to perform a large number of operations on a string, such as replacing substrings or splitting the string into multiple substrings, it can be slow and consume a lot resources.
- Strings can be difficult to work with when you need to represent complex data structures, such as lists or dictionaries. In these cases, it may be more efficient to use a different data type, such as a list or a dictionary, to represent the data.

Operations

- Upper / Lower Case
- Remove white spaces
- Replace String
- Split String
- String Indexing
- String Slicing
- Format String